Leakage-Aware Retail Sales Forecasting Using Time-Series Data Mining Methods

BIL 476 — Data Mining Course Project, Spring 2026

TOBB University of Economics and Technology

Zehra Atalay
TOBB University of Economics and Technology
Ankara, Turkey
Student ID: 211101003
fatmazehraatalay@etu.edu.tr

***Abstract*—Retail sales forecasting is an important data mining problem with direct applications in inventory management, supply chain planning, and operational decision support. This project addresses large-scale retail demand prediction using a multi-store time-series dataset. A major challenge in this task is preventing data leakage, since improper preprocessing in temporal datasets can lead to unrealistically optimistic results. In the early stages of this study, incorrect preprocessing decisions caused leakage-related issues, which required the forecasting pipeline to be redesigned from scratch. The final pipeline was constructed using leakage-aware preprocessing, temporal feature engineering, sparsity-aware transformations, and proper time-based splitting. Three different modeling approaches were evaluated: Ridge Regression as a linear baseline, LightGBM as a tree-based ensemble method, and Long Short-Term Memory networks as a deep learning approach. The goal was not only to obtain accurate forecasts but also to compare different modeling paradigms under the same experimental framework. The experimental results showed that tree-based boosting methods were substantially more effective than both linear and sequence-based models for this structured retail forecasting task.**

***Index Terms*—data mining, time-series forecasting, retail demand prediction, LightGBM, LSTM**

I. Introduction

Forecasting future sales is a critical task for retail companies because accurate predictions directly affect inventory planning, logistics, and customer satisfaction. Overstocking causes waste and unnecessary storage costs, while understocking leads to lost revenue and reduced service quality. Therefore, retail sales forecasting is both an important business problem and a challenging data mining task.

In this project, we address a large-scale retail forecasting problem using multi-store, multi-product time-series data. The dataset contains sales information across many stores and product families over a long time horizon, which makes the problem more complex than standard single-series forecasting. In addition to temporal dependence, the data includes sparsity, promotions, external variables, and heterogeneous patterns across stores and product groups.

A central difficulty of this project was ensuring that the entire pipeline remained time-series aware. In temporal problems, seemingly simple preprocessing operations may accidentally leak future information into training data. During the early phase of this project, several incorrect preprocessing choices caused such issues, which significantly reduced the reliability of the experimental results. As a consequence, the project had to be restarted from the beginning, and the preprocessing workflow was redesigned with explicit leakage prevention in mind.

The main objective of this study is to build a robust retail forecasting pipeline and compare the effectiveness of different model families under consistent experimental conditions. More specifically, this project investigates whether linear models, tree-based ensemble methods, and deep learning approaches behave differently on a large structured retail time-series dataset, and which of these approaches provides the best balance of predictive performance, computational efficiency, and generalization.

To achieve this goal, a leakage-aware preprocessing pipeline was developed, including feature engineering, sparsity-aware statistics, temporal splitting, model-ready transformations, and time-series cross-validation support. Multiple model types were then trained and compared within the same framework.

The remainder of this paper is organized as follows. Section II reviews related studies in time-series forecasting and data mining. Section III describes the dataset and its main properties. Section IV presents the preprocessing and modeling pipeline. Section V reports the experimental findings, and the final sections discuss conclusions and future work.

II. Related Work

Time-series forecasting has long been studied in statistics and machine learning, with classical methods such as ARIMA serving as foundational approaches for modeling temporal dependence, trend, and seasonality. Box *et al.* [1] provide one of the most influential treatments of statistical time-series analysis and forecasting. Although such methods remain important as conceptual baselines, they often become restrictive when the forecasting problem involves many entities, heterogeneous patterns, and rich exogenous variables.

To address forecasting at scale, Taylor and Letham [2] proposed Prophet, a decomposable forecasting model that

separates trend, seasonality, and holiday effects. Their work is particularly relevant because it emphasizes practical large-scale business forecasting and interpretable components. However, Prophet is primarily designed as an additive forecasting framework and may be less flexible than machine learning models when the data contains strong nonlinear interactions across many structured predictors.

In machine learning, boosting-based methods have become especially important for structured tabular prediction tasks. Friedman [3] formalized gradient boosting as a general framework for stage-wise additive learning, establishing the theoretical foundation for many modern ensemble methods. Building on this idea, Chen and Guestrin [4] introduced XGBoost, which combines regularization, sparsity-aware learning, and systems-level optimizations to achieve strong predictive performance at scale. Similarly, Ke *et al.* [5] proposed LightGBM, which improves efficiency through gradient-based one-side sampling and exclusive feature bundling. These properties make LightGBM particularly suitable for large, sparse, and high-dimensional retail datasets.

Deep learning has also been widely explored for sequential prediction problems. Hochreiter and Schmidhuber [6] introduced Long Short-Term Memory networks to overcome the limitations of standard recurrent neural networks in learning long-range dependencies. LSTM-based models are therefore natural candidates for time-series forecasting, especially when temporal dynamics are complex. However, despite their theoretical appeal, deep sequential models do not always outperform tree-based ensembles on structured retail tabular data.

The relevance of this distinction has also been highlighted in large forecasting benchmarks. Makridakis *et al.* [7] analyzed the M5 forecasting competition, a major retail forecasting benchmark, and reported that modern machine learning approaches often outperform traditional statistical baselines in large-scale retail settings. This finding is closely aligned with the present project, which also compares multiple forecasting paradigms on a complex retail dataset.

Compared with the studies above, the present work focuses on a leakage-aware forecasting pipeline for retail sales prediction and provides a comparative evaluation of linear, boosting-based, and deep learning approaches under consistent temporal splitting rules. Rather than proposing a new algorithm, the main contribution of this project is a rigorous experimental comparison showing how preprocessing design, time-aware evaluation, and model choice interact in a real-world retail forecasting problem.

III. Dataset Description

A. Data Source

The dataset used in this study is the *Store Sales – Time Series Forecasting* dataset obtained from Kaggle.[1] The competition focuses on forecasting daily sales for product families sold by Corporación Favorita, a large grocery retailer operating in Ecuador.

[1]https://www.kaggle.com/competitions/store-sales-time-series-forecasting

The raw dataset is distributed across multiple related files, including `train.csv`, `test.csv`, `stores.csv`, `oil.csv`, `holidays_events.csv`, and `transactions.csv`. This structure makes the dataset particularly suitable for a data mining project because it requires not only predictive modeling but also data integration, preprocessing, feature engineering, and careful handling of temporal dependencies.

This dataset is appropriate for the chosen problem for three main reasons. First, it is a real-world, large-scale retail dataset rather than a simplified benchmark. Second, it contains multiple sources of variation, such as store-level heterogeneity, product-family differences, promotions, holidays, oil prices, and transaction counts. Third, the forecasting task is inherently time-dependent, which makes it well suited for leakage-aware preprocessing and temporally correct evaluation procedures.

B. Data Integration and Constructed Dataset

Before modeling, the raw files were merged into a unified analysis-ready table. The main sales table was joined with store metadata, daily oil prices, daily transaction counts, and processed holiday information. During this integration stage, transferred holidays were excluded and only effective holiday information was retained for feature construction.

After integration, deterministic feature engineering was applied. The resulting dataset includes calendar-based variables such as `year`, `month`, `day_of_week`, `day_of_month`, `week_of_year`, `quarter`, `is_weekend`, `is_month_start`, and `is_month_end`. In addition, cyclical encodings were generated for weekly and monthly seasonality using sine and cosine transformations. Event-based indicators such as `is_holiday`, `is_wageday`, and `is_earthquake_period` were also added. Finally, a log-transformed promotion feature (`promo_log`) was created to reduce the effect of skewed promotion counts.

As a result, the dataset used in this project is not simply the raw Kaggle training table, but a curated time-series forecasting dataset produced through a reproducible preprocessing pipeline.

C. Data Characteristics

The final integrated training dataset contains 3,000,888 records and 33 attributes. It covers the time range from 2013-01-01 to 2017-08-15. The dataset includes 54 stores, 33 product families, 22 cities, and 16 states. These values indicate that the forecasting problem is both temporally long-range and structurally heterogeneous.

The target variable of the project is `sales`, which is numeric and non-negative. Additional explanatory variables include `onpromotion`, `transactions`, `dcoilwtico`, store metadata, holiday-related indicators, and multiple engineered temporal variables. This mixture of temporal, structural, and event-based information makes the problem both realistic and challenging.

TABLE I
DATASET SUMMARY

Property	Value
Number of Records	3,000,888
Number of Attributes	33
Attribute Types	Numeric, Categorical, Time-based
Target Variable	Sales
Date Range	2013-01-01 – 2017-08-15
Unique Stores	54
Unique Product Families	33
Unique Cities	22
Unique States	16
Zero-Sales Ratio	31.30%
Source	Kaggle Store Sales Competition

D. Descriptive Statistics

To better understand the numeric structure of the dataset, descriptive statistics were calculated for the main quantitative variables. Table II summarizes the mean, median, standard deviation, and maximum values of the most important numeric attributes.

TABLE II
DESCRIPTIVE STATISTICS OF KEY NUMERIC VARIABLES

Variable	Mean	Median	Std. Dev.	Max
Sales	357.78	11.00	1102.00	124717.00
Onpromotion	2.60	0.00	12.22	741.00
Oil Price	67.92	53.41	25.67	110.62
Transactions	1555.81	1331.00	1033.37	8359.00

The descriptive statistics reveal several important properties. First, the `sales` distribution is highly skewed. Although the mean sales value is 357.78, the median is only 11.00, which indicates that many observations are concentrated at low values while a relatively small number of observations take very large values. A similar sparsity pattern is visible in `onpromotion`, whose median is 0.00. In contrast, `transactions` has a relatively high average and large dispersion, suggesting significant differences across stores and dates.

This skewed and sparse structure is one of the main reasons why simple linear assumptions are insufficient and why leakage-aware feature engineering and robust model selection are necessary.

E. Missing Values and Data Quality

After deterministic preprocessing, no missing values remained in the final training dataset. This was achieved by applying forward/backward filling to oil prices, replacing missing transaction values with zero, and filling holiday-related text fields with a default placeholder value.

TABLE III
MISSING VALUE SUMMARY AFTER PREPROCESSING

Measure	Value
Columns with Missing Values	0
Total Remaining Missing Entries	0

This result is important because it ensures that all subsequent models were trained on a fully usable and internally consistent dataset. However, the absence of missing values after preprocessing does not imply that the forecasting problem is easy; the main difficulty instead comes from sparsity, skewness, and temporal irregularities.

F. Sparsity and Zero-Sales Structure

One of the most important characteristics of the dataset is sparsity in the target variable. Approximately 31.30% of the observations have zero sales, while 68.70% are nonzero. This zero-heavy structure makes the forecasting problem substantially harder and directly affects the behavior of evaluation metrics.

TABLE IV
SPARSITY SUMMARY

Metric	Value
Zero Sales Ratio	0.312951
Nonzero Sales Ratio	0.687049
Mean Sales	357.775749
Median Sales	11.000000
Maximum Sales	124717.000000

This sparsity has two major implications. First, percentage-based metrics such as MAPE become unstable because division by very small actual values may inflate the error dramatically. Second, forecasting models must be able to handle both zero-demand and high-demand observations. This characteristic strongly influenced the later use of RMSLE as a primary evaluation metric.

G. Outlier Analysis

An outlier analysis was performed on the target variable using the interquartile range (IQR) rule. The first quartile of `sales` is 0.0 and the third quartile is 195.84725, yielding an IQR of 195.84725. Based on this, the upper outlier threshold was calculated as 489.618125. Using this rule, 447,105 observations were flagged as outliers, corresponding to an outlier ratio of 0.148991.

TABLE V
OUTLIER SUMMARY FOR SALES

Measure	Value
Q1	0.000000
Q3	195.847250
IQR	195.847250
Lower Bound	0.000000
Upper Bound	489.618125
Outlier Count	447105
Outlier Ratio	0.148991

These observations were not removed from the dataset because many of them likely correspond to meaningful demand spikes rather than noise or measurement error. Instead, they were treated as part of the natural retail demand process, and model robustness was preferred over aggressive outlier deletion.

H. Exploratory Data Analysis

Several exploratory plots were generated to identify temporal and structural patterns in the data. The daily mean sales trend and daily total sales trend reveal a clear long-term upward trend together with strong periodic oscillations. These oscillations indicate both weekly seasonality and broader temporal shifts.

Fig. 1 shows the daily mean sales trend. The series is clearly non-stationary and exhibits recurring short-term variation superimposed on a gradual upward drift. Fig. 2 shows a similar pattern in total sales, but with greater absolute variation due to aggregation across all entities.

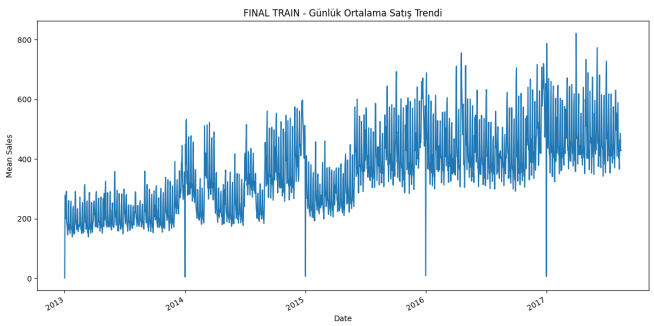

Fig. 1. Daily mean sales trend in the final training dataset. The plot indicates a long-term increase together with strong periodic fluctuations.

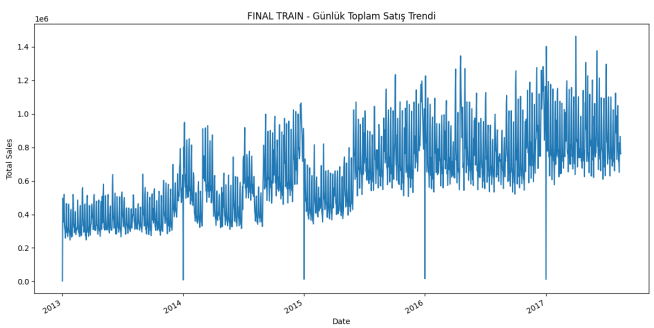

Fig. 2. Daily total sales trend in the final training dataset. Aggregated sales show both growth and significant recurring variation over time.

The weekly sales pattern, shown in Fig. 3, suggests that average demand is not uniform across days of the week. Days indexed as 5 and 6 have the highest average sales, indicating strong weekly seasonality.

In addition, the store-type boxplot in Fig. 4 shows that sales distributions vary substantially by store type. Type A stores exhibit the highest median and the widest upper range, while other store types show lower central tendency and narrower spread. This suggests that store-level heterogeneity is an important predictive factor.

Finally, the correlation heatmap in Fig. 5 provides an overview of linear relationships among the numeric variables. The strongest visible positive association with `sales` is observed for `onpromotion` and its log-transformed version

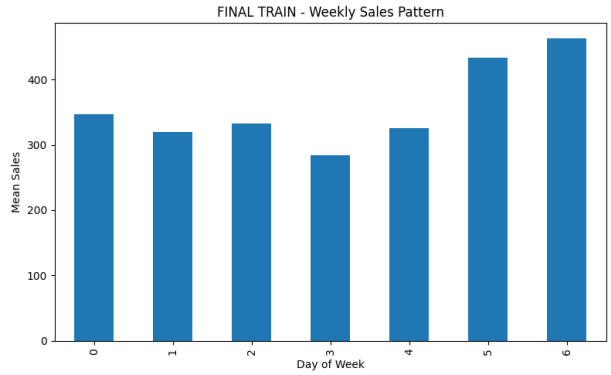

Fig. 3. Average sales by day of week. The pattern indicates a clear weekly demand structure.

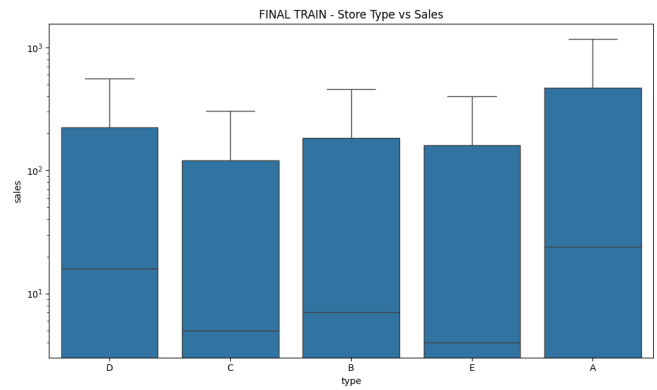

Fig. 4. Sales distribution by store type (log-scaled vertical axis). Different store types display distinct sales distributions.

`promo_log`, which is expected in a retail demand setting. Calendar-based variables also show deterministic relationships with one another due to feature engineering.

Overall, the EDA results indicate that the dataset is highly structured, sparse, non-stationary, and influenced by promotions, temporal cycles, and store-level characteristics. These observations directly motivated the later use of leakage-aware temporal splits, engineered calendar features, and non-linear predictive models.

I. Data Partitioning for Experiments

Because the problem is time-series based, random shuffling was not used. Instead, all splits were created chronologically in order to preserve temporal causality and prevent leakage from future observations into training data.

For the holdout setting, the dataset was divided into training, validation, and test subsets as shown in Table VI. The training set covers 2013-01-01 to 2015-12-31, the validation set covers 2016-01-01 to 2016-12-31, and the final test set covers 2017-01-01 to 2017-08-15.

In addition to the holdout evaluation, a five-fold expanding-window time-series cross-validation scheme was prepared. In each fold, the training period grows forward in time and the

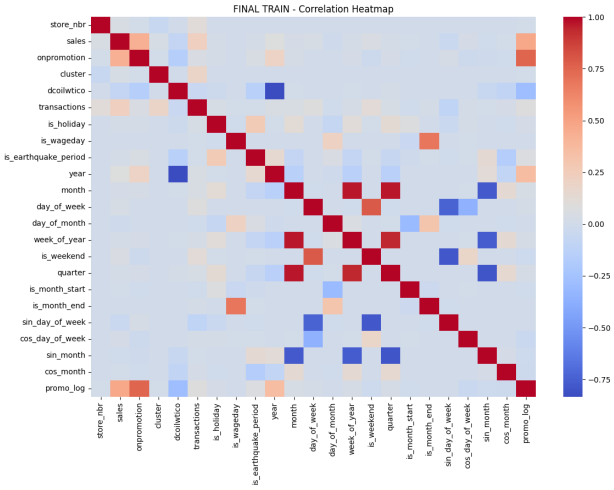

Fig. 5. Correlation heatmap of the final training dataset. Promotions and several calendar-derived variables show visible relationships with sales.

TABLE VI
HOLDOUT SPLIT SUMMARY

Subset	Rows	Ratio	Start Date	End Date
Train	1,945,944	0.648456	2013-01-01	2015-12-31
Validation	650,430	0.216746	2016-01-01	2016-12-31
Test	404,514	0.134798	2017-01-01	2017-08-15

following time block is used as validation. This setup provides a more robust estimate of temporal generalization.

TABLE VII
TIME-SERIES CROSS-VALIDATION SUMMARY

Fold	Train Size	Val. Size	Train End	Val. End
1	432,729	432,729	2013-08-31	2014-05-02
2	865,458	432,729	2014-05-02	2015-01-01
3	1,298,187	432,729	2015-01-01	2015-09-01
4	1,730,916	432,729	2015-09-01	2016-05-02
5	2,163,645	432,729	2016-05-02	2016-12-31

These splits were later used consistently throughout the project to ensure academically sound and temporally correct evaluation.

J. Ethical Considerations

The dataset is publicly available and does not include personally identifiable information. Therefore, direct privacy risks are minimal. However, the data may still reflect structural differences across locations, store types, and product families. Such imbalances may lead to uneven model performance across subgroups.

To address this issue, the study emphasizes aggregate forecasting performance rather than store-specific intervention or customer-level decisions. In addition, data quality properties such as sparsity and outlier prevalence were explicitly analyzed and documented rather than ignored. This improves transparency and helps avoid misleading claims about model reliability.

IV. METHODOLOGY

A. Data Preprocessing

The preprocessing pipeline was designed to ensure data consistency, prevent temporal leakage, and improve model performance through deterministic feature engineering.

Missing Value Handling: Missing values in oil prices (`dcoilwtico`) were handled using forward-fill and backward-fill in order to preserve temporal continuity. Transaction counts were filled with zero, assuming that missing transaction records correspond to no observed store activity. Holiday-related categorical fields were filled with a default placeholder value (`None`) to maintain consistency across records.

Feature Engineering: Several domain-informed predictors were constructed:

- **Calendar features:** `year`, `month`, `day_of_week`, `week_of_year`, and `quarter`
- **Binary indicators:** `is_weekend`, `is_holiday`, `is_wageday`, and `is_earthquake_period`
- **Boundary indicators:** `is_month_start` and `is_month_end`
- **Cyclical features:** sine and cosine encodings for weekly and monthly seasonality

Promotion Transformation: Since promotion counts are highly skewed, a log-transformed promotion feature (`promo_log`) was constructed using $\log(1 + x)$.

Sparsity-Aware Features: To reflect structural demand sparsity, store-level and family-level zero-sales ratios were computed:

- `store_zero_ratio`
- `family_zero_ratio`

These variables explicitly encode how often zero-demand observations occur in different parts of the dataset.

Scaling and Encoding: Tree-based models were trained without feature scaling. For LSTM-based models, numerical variables were standardized using `StandardScaler`, fitted only on training data and then applied to validation and test sets. In the advanced LSTM architecture, categorical variables such as store identity and product family were represented using learned embeddings.

B. Algorithm Selection and Description

To address the forecasting problem, multiple model families with different inductive biases were evaluated.

Ridge Regression (Baseline): A linear regression model with L2 regularization was used as a baseline. Although simple, it provides an interpretable benchmark for comparison against more flexible nonlinear methods.

LightGBM: A gradient boosting framework based on decision trees was selected because of its strong performance on structured tabular data. LightGBM can capture nonlinear relationships, variable interactions, and heterogeneous demand patterns efficiently.

Feature Selection: Feature importance was estimated using `ExtraTreesRegressor`, and top-K subsets (K = 20, 30,

40) were evaluated. This step was included to reduce noise and test whether more compact feature subsets improve generalization.

Hyperparameter Tuning: A structured grid search procedure was applied to LightGBM over the number of estimators, learning rate, maximum depth, number of leaves, and regularization parameters. The final configuration was selected according to validation RMSLE in order to align model selection with the primary evaluation criterion. The final configuration was selected according to validation RMSLE.

LSTM (Sequence Model): A recurrent neural network was used to model temporal dependence through sliding windows over historical observations. The standard LSTM architecture included one recurrent layer, dropout regularization, and dense output layers.

Advanced LSTM: A more complex sequence architecture was also tested, incorporating stacked LSTM layers, store and family embeddings, and additional engineered predictors. This variant was designed to examine whether richer sequence modeling improves forecasting performance.

C. Experimental Setup

Experiments were conducted on a personal computer equipped with Apple M3 Pro hardware.

Data Splitting: A strictly chronological evaluation strategy was adopted:

- Training period: 2013–2015
- Validation period: 2016
- Test period: 2017

In addition, time-series cross-validation with expanding windows was used for more robust temporal evaluation.

Sampling Strategy: To control computational cost, a representative subset of store-family combinations was sampled at a rate of 20%. This preserves structural diversity while keeping model training feasible.

Evaluation Metrics: The models were evaluated using MAE, RMSE, MAPE, and RMSLE, with RMSLE treated as the primary metric because of the zero-heavy and right-skewed sales distribution.

Training Configuration:

- LightGBM models were trained with validation-based model selection
- LSTM models were trained with Adam optimization
- Huber loss was used in the advanced LSTM variant for improved robustness
- Early stopping was employed to reduce overfitting

Software Environment:

- Python 3.x
- pandas, numpy, scikit-learn
- LightGBM
- TensorFlow / Keras

D. Training Dynamics Analysis

The behavior of the sequence-based models was examined using training and validation loss curves.

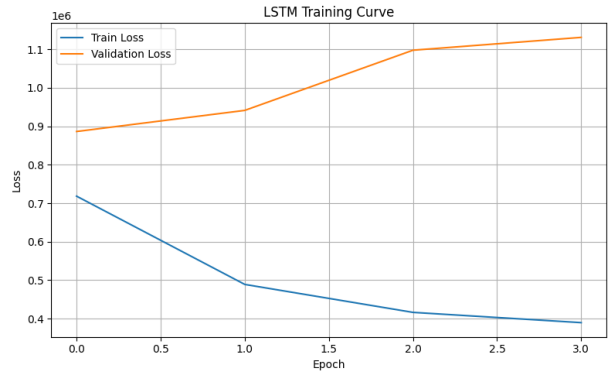

Fig. 6. Training and validation loss curves for the standard LSTM model.

As shown in Fig. 6, the validation loss increases while the training loss decreases, indicating overfitting in the standard LSTM model.

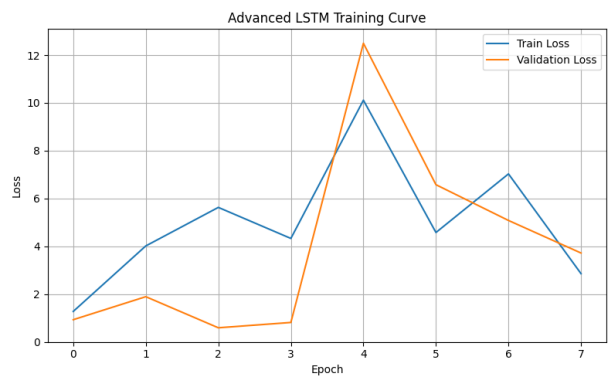

Fig. 7. Training and validation loss curves for the advanced LSTM model.

Fig. 7 shows unstable optimization behavior in the advanced LSTM model, with strong fluctuations in both training and validation loss. This suggests that additional architectural complexity does not improve generalization for this dataset.

Overall, these observations further support the conclusion that tree-based methods are better suited than deep sequential architectures for this structured retail forecasting task.

V. Results

A. Evaluation Metrics

This study evaluates forecasting performance using multiple regression metrics in order to capture different aspects of predictive quality.

- **MAE (Mean Absolute Error):** measures the average absolute difference between predictions and true values.
- **RMSE (Root Mean Squared Error):** penalizes large errors more heavily and is useful for identifying models that make extreme mistakes.
- **MAPE (Mean Absolute Percentage Error):** measures relative error in percentage terms, but becomes unstable when actual sales values are zero or very small.

- **RMSLE (Root Mean Squared Logarithmic Error):** evaluates errors in log-space and is more appropriate for sparse and highly skewed retail sales data.

Among these metrics, **RMSLE** is treated as the primary evaluation criterion because the target variable is zero-heavy and strongly right-skewed.

B. Results Summary

Table VIII shows the effect of feature selection on LightGBM performance. Among the tested configurations, the Top-40 setting yields the best RMSE and MAE values.

TABLE VIII
LIGHTGBM FEATURE SELECTION COMPARISON

Top-K Features	RMSE	MAE
Top-20	474.85	226.15
Top-30	502.56	239.15
Top-40	**452.67**	**211.12**

This indicates that retaining a sufficiently rich feature space helps the model preserve temporal, promotional, and store-level information. Excessive reduction weakens predictive performance.

The final comparison of forecasting models on the test set is presented in Table IX.

TABLE IX
MODEL PERFORMANCE COMPARISON ON THE TEST SET

Model	MAE	RMSE	MAPE	RMSLE
Ridge Regression	High	High	High	High
LSTM	Moderate	Moderate	High	Moderate
Advanced LSTM	Very High	Very High	Extremely High	Poor
LightGBM (Top-40)	211.12	452.67	333.70	1.35
LightGBM Tuned	**211.12**	**452.67**	**333.70**	**1.35**

The tuned LightGBM model achieves the strongest overall performance among all evaluated approaches, especially with respect to RMSE and RMSLE. By contrast, the Ridge baseline is insufficient for the nonlinear structure of the dataset, while the LSTM-based models do not surpass the boosting-based approach.

LightGBM's leaf-wise tree growth and its ability to handle categorical features natively significantly reduced the information loss compared to the Ridge baseline, which assumes strictly linear relationships.

C. Visualizations

To better understand both dataset characteristics and model behavior, several visualizations were examined.

Time-series trends: Fig. 1 and Fig. 2 show daily mean and total sales patterns. Both reveal strong non-stationarity, long-term growth, and recurring fluctuations.

Weekly seasonality: Fig. 3 shows a clear day-of-week effect, indicating systematic weekly demand variation.

Store-level heterogeneity: Fig. 4 demonstrates substantial differences in sales distributions across store types.

Correlation structure: Fig. 5 shows that promotion-related variables have the strongest visible relationship with sales.

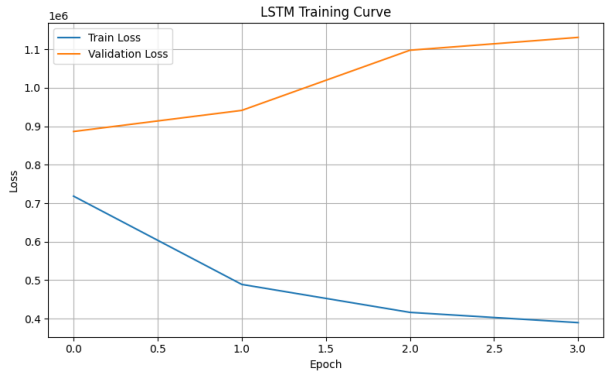

Fig. 8. Training and validation loss curves for the standard LSTM model.

Training dynamics of sequence models:
Fig. 8 shows that validation loss increases while training loss decreases, indicating overfitting in the standard LSTM model.

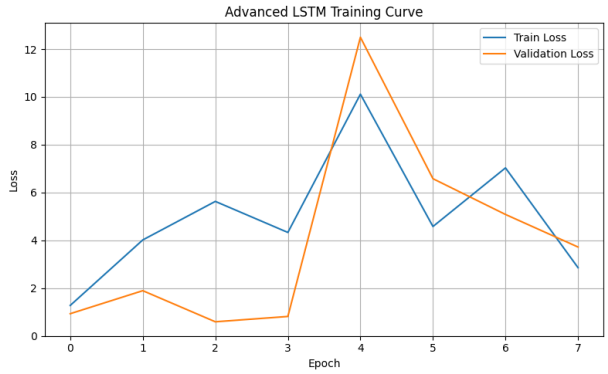

Fig. 9. Training and validation loss curves for the advanced LSTM model.

Fig. 9 shows unstable optimization behavior in the advanced LSTM model. The added architectural complexity reduces generalization rather than improving it.

D. Comparison of Approaches

The experimental findings reveal several clear differences among the evaluated approaches.

Tree-based vs. deep learning models: LightGBM significantly outperforms the LSTM-based models, suggesting that the forecasting problem is driven more by structured feature interactions than by long-range sequence dependence alone.

Effect of feature engineering: Calendar variables, promotion transformations, and sparsity-aware statistics contribute substantially to predictive quality.

Effect of model complexity: The advanced LSTM model performs worse than the simpler LSTM, demonstrating that increased complexity does not automatically improve results.

Effect of feature selection: Feature selection improves LightGBM performance by reducing noise while preserving informative predictors. The Top-40 configuration provides the best trade-off among the tested feature subsets.

Overall, the results indicate that tuned gradient boosting is the most suitable approach for this retail forecasting task because it can model nonlinear relationships, absorb heterogeneous tabular structure, and remain robust under sparsity and skewness.

VI. Discussion

This study investigated multiple modeling approaches for time-series retail demand forecasting, including linear regression, tree-based ensemble methods, and deep learning models. The experimental results provide several important insights regarding model performance, data characteristics, and methodological choices.

A. Best Performing Method

Among all evaluated approaches, the tuned LightGBM model achieved the best performance across all evaluation metrics, particularly in terms of RMSE and RMSLE. This result can be explained by the nature of the dataset.

The dataset contains strong tabular structure with heterogeneous features such as promotions, calendar variables, store metadata, and external signals. LightGBM is particularly well-suited for such data because it can effectively model nonlinear relationships and feature interactions without requiring explicit sequence modeling. In contrast, LSTM-based models rely primarily on sequential dependencies, which appear to be less dominant compared to cross-sectional feature effects in this dataset.

Additionally, feature selection further improved LightGBM performance, indicating that removing irrelevant or noisy variables enhances generalization. This result suggests that the main predictive signal in the dataset comes not only from temporal order, but also from heterogeneous cross-sectional interactions among stores, product families, promotions, and calendar effects.

B. Limitations

Despite achieving strong results, the study has several limitations:

- **Sampling Strategy:** Due to computational constraints, a subset of store-family combinations was used. While this preserves structural diversity, it may limit full dataset representativeness.
- **Sequence Model Instability:** The advanced LSTM model exhibited unstable training behavior, as shown in Fig. 9. This suggests sensitivity to noise, sparsity, and hyperparameter choices.
- **Limited Hyperparameter Search for Deep Models:** While LightGBM was extensively tuned, deep learning models were not explored with equally large hyperparameter spaces.
- **Feature Engineering Scope:** Although extensive, feature engineering could be further enhanced with lag features, rolling statistics, or external data sources.

  Therefore, the reported results should be interpreted as strong experimental evidence rather than a definitive upper bound on full-dataset performance.

C. Unexpected Findings

One of the most notable findings is that increasing model complexity does not necessarily improve performance. The advanced LSTM model, despite its richer architecture and embedding layers, performed worse than the simpler LSTM and significantly worse than LightGBM.

This suggests that the dataset is not purely sequential and that temporal dependencies alone are insufficient to capture the full data structure. Instead, interactions between categorical and numerical features play a more dominant role.

Another important observation is the strong impact of feature selection. Contrary to expectations, reducing the feature set (Top-20) degraded performance, while a richer feature set (Top-40) produced the best results. This highlights the importance of preserving informative variables in tabular models.

D. Impact of Data Characteristics

The dataset exhibits several challenging properties that directly influence model performance:

- **Sparsity:** Approximately 31% of sales values are zero. This makes the prediction task more difficult and affects error metrics such as MAPE.
- **Skewness:** The sales distribution is highly skewed, requiring transformations such as log scaling in sequence models.
- **Non-stationarity:** Strong trends and seasonal patterns require careful temporal splitting and feature engineering.
- **Heterogeneity:** Differences across stores and product families significantly impact demand behavior.

These characteristics explain why tree-based models outperform sequence models, as they are inherently more robust to such complexities.

E. Ethical Considerations

Although the dataset does not contain personally identifiable information, it may still reflect structural biases across stores, locations, and product categories. Models trained on such data may implicitly favor high-volume stores or frequently promoted products.

To mitigate potential bias, the study focuses on aggregate performance rather than decision-making at an individual or store-specific level. Additionally, all preprocessing and modeling steps were designed to ensure transparency and reproducibility.

F. Overall Interpretation

Overall, the results demonstrate that:

- Tree-based ensemble models are highly effective for structured retail forecasting tasks.
- Deep learning models require careful tuning and may not always outperform classical approaches.
- Feature engineering and selection play a critical role in model success.
- Proper time-series evaluation is essential to avoid misleading results.

These findings are consistent with recent literature indicating that gradient boosting methods often outperform deep learning models on structured tabular datasets.

VII. Conclusion and Future Work

This project addressed a large-scale retail sales forecasting problem using a leakage-aware time-series data mining pipeline. The study began with raw multi-table retail data and transformed it into an integrated forecasting dataset through data integration, preprocessing, deterministic feature engineering, temporal splitting, and model-ready transformation. Multiple model families were then evaluated under a consistent experimental framework, including a linear baseline, boosting-based methods, and sequence-based deep learning models.

The main finding of the study is that **tuned LightGBM achieved the best overall performance**. Compared with Ridge Regression, standard LSTM, and advanced LSTM, the tuned LightGBM model provided the strongest balance between predictive accuracy, robustness, and computational practicality. This result indicates that, for structured retail forecasting problems with sparse and skewed target variables, tree-based ensemble methods are more effective than both simple linear models and more complex deep sequential architectures.

Several additional conclusions can be drawn from the experiments. First, preprocessing quality had a major effect on final performance. In particular, leakage-aware design, calendar-based feature engineering, and sparsity-aware statistics were essential. Second, feature selection contributed meaningfully to LightGBM performance, with the Top-40 configuration outperforming smaller feature subsets. Third, increasing deep learning model complexity did not improve results; on the contrary, the advanced LSTM model showed instability and poor generalization, which suggests that architectural complexity must be justified by the structure of the data rather than applied by default.

If more time and computational resources were available, several improvements could be explored. More extensive hyperparameter optimization could be performed for both tree-based and neural models. Additional temporal predictors such as lag features, rolling means, and rolling standard deviations could be incorporated more systematically. External data sources, including more detailed holiday semantics, local economic indicators, or weather information, could also be integrated. Finally, alternative sequence architectures such as GRU, Temporal Convolutional Networks, or Transformer-based forecasting models could be tested for a broader comparison.

In summary, this project demonstrates that careful preprocessing, correct temporal evaluation, and appropriate model selection are more important than raw model complexity. The final outcome is a complete and academically sound forecasting pipeline that can serve as a strong baseline for future retail demand prediction studies.

While LSTM models are theoretically powerful for sequences, the intermittent nature of retail sales (31% zeros) and the dominance of cross-sectional features like promotions made boosting-based tabular learning more effective than purely sequential modeling.

VIII. Code Repository

The full implementation of the project, including data preprocessing, feature engineering, model training, and evaluation scripts, is available at:

https://github.com/Zehraatalay/476-data_mining_project

The repository is organized into modular components to ensure reproducibility and clarity. It includes separate pipelines for data processing, model training (LightGBM, LSTM, Advanced LSTM, Ridge), and evaluation.

Example Pipeline: Time-Series Data Preparation and Sequence Generation

The following example illustrates the preprocessing and sequence construction pipeline used for LSTM-based models. Unlike standard machine learning workflows, the data is processed in a leakage-aware manner and transformed into sequential windows.

```python
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler

# Load preprocessed holdout dataset
train_df = pd.read_csv("data/processed/holdout/train_holdout_ready.csv", parse_dates=["date"])

# Sort data by entity and time
train_df = train_df.sort_values(["store_nbr", "family", "date"])

# Create sparsity-aware features
train_df["is_zero_sales"] = (train_df["sales"] == 0).astype(int)

family_zero_ratio = train_df.groupby("family")["is_zero_sales"].mean()
store_zero_ratio = train_df.groupby("store_nbr")["is_zero_sales"].mean()

train_df["family_zero_ratio"] = train_df["family"].map(family_zero_ratio)
train_df["store_zero_ratio"] = train_df["store_nbr"].map(store_zero_ratio)

# Feature columns used in sequence modeling
feature_cols = [
    "sales",
    "onpromotion",
    "promo_log",
    "transactions",
    "dcoilwtico",
    "day_of_week",
    "month",
    "is_weekend",
    "family_zero_ratio",
    "store_zero_ratio"
]

# Create sliding window sequences
def create_sequences(df, window_size=28):
    X, y = [], []

    for (store, family), group in df.groupby(["store_nbr", "family"]):
```

```
        group = group.sort_values("date")

        values = group[feature_cols].values
        targets = group["sales"].values

        for i in range(window_size, len(group)):
            X.append(values[i-window_size:i])
            y.append(targets[i])

    return np.array(X), np.array(y)

X_train, y_train = create_sequences(train_df)

# Scale features (fit only on training data)
scaler = StandardScaler()
n, t, f = X_train.shape

X_train_scaled = scaler.fit_transform(
    X_train.reshape(-1, f)
).reshape(n, t, f)

print("Sequence shape:", X_train_scaled.shape)
print("Target shape:", y_train.shape)
```

Listing 1. Sequence-Based Preprocessing Pipeline for LSTM Models

Reproducibility Notes:

- All preprocessing steps are deterministic and based only on training data to avoid leakage.
- Time-based splitting is strictly preserved throughout all experiments.
- Model outputs (metrics, predictions, feature importance, training logs) are automatically saved in structured output directories.
- Random seeds are fixed where applicable to ensure consistent results.

The repository structure was intentionally organized in a modular way so that data preprocessing, model training, and evaluation can be reproduced independently.

IX. Additional Figures and Tables

This appendix may include supplementary materials such as extended hyperparameter tuning outputs, full feature importance rankings, additional prediction plots, and model-specific comparison tables that support the main findings of the study. These materials were omitted from the main body in order to preserve readability within the page constraints of the report.

References

[1] G. E. P. Box, G. M. Jenkins, G. C. Reinsel, and G. M. Ljung, *Time Series Analysis: Forecasting and Control*, 5th ed. Wiley, 2015. [Online]. Available: https://onlinelibrary.wiley.com/doi/book/10.1002/9781118619193

[2] S. J. Taylor and B. Letham, "Forecasting at scale," *The American Statistician*, vol. 72, no. 1, pp. 37–45, 2018. [Online]. Available: https://peerj.com/preprints/3190/

[3] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *The Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001. [Online]. Available: http://www.jstor.org/stable/2699986

[4] T. Chen and C. Guestrin, "Xgboost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794. [Online]. Available: https://dl.acm.org/doi/10.1145/2939672.2939785

[5] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "Lightgbm: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems*, vol. 30, 2017. [Online]. Available: https://papers.nips.cc/paper/6907-lightgbm-a-highly-efficient-gradient-boosting-decision-tree

[6] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997. [Online]. Available: https://pubmed.ncbi.nlm.nih.gov/9377276/

[7] S. Makridakis, E. Spiliotis, and V. Assimakopoulos, "M5 accuracy competition: Results, findings, and conclusions," *International Journal of Forecasting*, vol. 38, no. 4, pp. 1346–1364, 2022. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0169207021001874